



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
образовательное учреждение высшего образования
«Московский государственный технический
университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЛАБОРАТОРНАЯ РАБОТА 3

«Функциональные возможности языка Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков
программирования»

Выполнил: студент гр. ИУ5-34Б

(Подпись) (Олейник В. Д.)
(Ф.И.О.)

Проверил:

(Подпись) (Гапанюк Ю. Е.)
(Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

2026 г.

Задание 1: field.py

Необходимо реализовать генератор field. Генератор последовательно выдает значения ключей словаря.

```
2 goods = [  
3     {'title': 'Ковер', 'price': 2000, 'color': 'green'},  
4     {'title': 'Диван для отдыха', 'color': 'black'}  
5 ]  
6  
7 def field(items, *args):  
8     assert len(args) > 0  
9  
10    if len(args) == 1:  
11        key = args[0]  
12  
13        for item in items:  
14            value = item.get(key)  
15  
16            if value is not None:  
17                yield value  
18  
19    else:  
20        for item in items:  
21  
22            filtered = {key: item.get(key) for key in args if item.get(key) is not None}  
23  
24            if filtered:  
25                yield filtered
```

Рис. 1 – реализация генератора

```
Тест: Один аргумент  
Ковер  
Диван для отдыха  
  
Тест: Несколько аргументов ('title', 'price')  
{'title': 'Ковер', 'price': 2000}  
{'title': 'Диван для отдыха'}
```

Рис. 2 – демонстрация работы генератора

Задание 2: gen_random.py

Необходимо реализовать генератор gen_random(количество, минимум, максимум), который последовательно выдает заданное количество случайных чисел в заданном диапазоне от минимума до максимума, включая границы диапазона.

```

1 import random
2 from collections.abc import Generator
3
4
5 def gen_random(num_count: int, begin: int, end: int) -> Generator[int, None, None]:
6
7     for _ in range(num_count):
8
9         yield random.randint(begin, end)
10

```

Рис. 3 – реализация генератора случайных чисел

```

(venv) C:\Users\user> python unique.py
Генерируем 10 случайных чисел от 10 до 50:
Получено число: 43
Получено число: 46
Получено число: 40
Получено число: 41
Получено число: 17
Получено число: 17
Получено число: 42
Получено число: 39
Получено число: 17
Получено число: 42
Тест пройден

```

Рис. 4 – демонстрация работы генератора случайных чисел

Задание 3: unique.py

Необходимо реализовать итератор, который принимает на вход массив или генератор и итерируется по элементам, пропуская дубликаты.

- Конструктор итератора также принимает на вход именованный bool-параметр `ignore_case`, в зависимости от значения которого будут считаться одинаковыми строки в разном регистре. По умолчанию этот параметр равен `False`
- При реализации необходимо использовать конструкцию `**kwargs`
- Итератор должен поддерживать работу как со списками, так и с генераторами
- Итератор не должен модифицировать возвращаемые значения

```

1  from collections.abc import Generator
2  from typing import Any
3
4  from gen_random import gen_random
5
6
7  class Unique(object):
8
9
10     def __init__(self, items: list[Any] | Generator[Any], **kwargs):
11
12         self.items = iter(items)
13
14         self.ignore_case: bool = kwargs.get('ignore_case', False)
15
16         self.seen: set[Any] = set()

```

```

7  class Unique(object):
18     def __next__(self):
19
20         item = next(self.items)
21
22         if self.ignore_case:
23
24             if isinstance(item, str):
25
26                 if item.lower() not in self.seen:
27                     self.seen.add(item.lower())
28
29                 return item
30
31             else:
32
33                 if item not in self.seen:
34                     self.seen.add(item)
35
36                 return item
37
38         else:
39
40             if item not in self.seen:
41                 self.seen.add(item)
42
43             return item
44

```

Рис. 6-5 – реализация итератора

```

--- Тест 1: Строки
Вход: ['яблоко', 'ЯБЛОКО', 'банан', 'яблоко', 'Банан']
Без учета регистра (ignore_case=False):
яблоко
ЯБЛОКО
банан
Банан

С учетом регистра (ignore_case=True):
яблоко
банан

--- Тест 2: Числа
Вход: [1, 2, 2, 3, 1, 4, 3, 5]
Только уникальные числа:
1
2
3
4
5

--- Тест 3: Итератор (генератор) ---
Уникальные числа из случайного генератора:
3
2
4
5
1

```

Рис. 7 – демонстрация работы итератора

Задание 4: sort.py

Дан массив 1, содержащий положительные и отрицательные числа.

Необходимо одной строкой вывести на экран массив 2, который содержит значения массива 1, отсортированные по модулю в порядке убывания.

```

def sort_w_lambda(items: list[int]) -> None:
    items.sort(key=lambda x: abs(x), reverse=True)

def sort_without_lambda(items: list[int]) -> None:
    items.sort(key=abs, reverse=True)

```

Рис. 8 – реализация сортировки

Задание 5: print_result.py

Необходимо реализовать декоратор print_result, который выводит на экран результат выполнения функции.

- Декоратор должен принимать на вход функцию, вызывать её, печатать в консоль имя функции и результат выполнения, после чего возвращать результат выполнения.
- Если функция вернула список, то значения элементов списка должны выводиться в столбик

- Если функция вернула словарь, то ключи и значения должны выводиться в столбик через знак равенства.

```
def print_result(func: Callable[..., Any]) -> Callable[..., Any]:  
    def wrapper(*args: Any, **kwargs: Any) -> Any:  
        result = func(*args, **kwargs)  
        print(func.__name__)  
        if isinstance(result, list):  
            for item in result:  
                print(f"{item}")  
        elif isinstance(result, dict):  
            for key, value in result.items():  
                print(f"{key} = {value}")  
        else:  
            print(result)  
        return result  
    return wrapper
```

Рис. 9 – реализация декоратора

```
!!!!!!!  
test_1  
1  
test_2  
iu5  
test_3  
a = 1  
b = 2  
test_4  
1  
2
```

Рис. 10 – реализация декоратора

Задание 6: cm_timer.py

Необходимо написать контекстные менеджеры cm_timer_1 и cm_timer_2, которые считают время работы блока кода и выводят его на экран.

```
class cm_timer_1():

    def __init__(self):

        self.start_time = 0.0

    def __enter__(self) -> Any:

        self.start_time = time.time()

    def __exit__(self, exc_type: Any, exc_val: Any, exc_tb: Any) -> None:

        end_time = time.time()

        elapsed_time = end_time - self.start_time

        print(f'time: {elapsed_time}')
```

Рис. 11 – реализация таймера на основе класса

```
26 @contextmanager
27 def cm_timer_2() -> Generator[None, Any, None]:
28
29     start_time = time.time()
30
31     try:
32
33         yield
34
35     finally:
36
37         end_time = time.time()
38
39         elapsed_time = end_time - start_time
40
41         print(f'time: {elapsed_time}')
```

Рис. 12 – реализация таймера на основе контекстного менеджера

```
(.venv) Vladimirs-Mac-mini:term3-PCPL-lab3-4 vladimir$ python cm_timer.py
--- Тест 1: Класс cm_timer_1
time: 1.5035598278045654

--- Тест 2: Функция cm_timer_2
time: 2.0051119327545166
(.venv) Vladimirs-Mac-mini:term3-PCPL-lab3-4 vladimir$ █
```

Рис. 13 – демонстрация работы таймера

Задание 7: process_data.py

В предыдущих задачах были написаны все требуемые инструменты для работы с данными. Применим их на реальном примере.

- В файле data_light.json содержится фрагмент списка вакансий.
- Структура данных представляет собой список словарей с множеством полей: название работы, место, уровень зарплаты и т.д.
- Необходимо реализовать 4 функции - f1, f2, f3, f4. Каждая функция вызывается, принимая на вход результат работы предыдущей. За счет декоратора @print_result печатается результат, а контекстный менеджер cm_timer_1 выводит время работы цепочки функций.
- Предполагается, что функции f1, f2, f3 будут реализованы в одну строку. В реализации функции f4 может быть до 3 строк.
- Функция f1 должна вывести отсортированный список профессий без повторений (строки в разном регистре считать равными). Сортировка должна игнорировать регистр. Используйте наработки из предыдущих задач.
- Функция f2 должна фильтровать входной массив и возвращать только те элементы, которые начинаются со слова “программист”. Для фильтрации используйте функцию filter.
- Функция f3 должна модифицировать каждый элемент массива, добавив строку “с опытом Python” (все программисты должны быть знакомы с Python). Пример: Программист C# с опытом Python. Для модификации используйте функцию map.
- Функция f4 должна сгенерировать для каждой специальности зарплату от 100 000 до 200 000 рублей и присоединить её к названию специальности. Пример: Программист C# с опытом Python, зарплата 137287 руб. Используйте zip для обработки пары специальность - зарплата.

```

2 import json
3 import sys
4 from typing import Any
5
6 from cm_timer import cm_timer_1
7 from gen_random import gen_random
8 from unique import Unique
9
10
11 def f1(data: list[dict[Any]]) -> list[str]:
12     return sorted(list(Unique((job['job-name'] for job in data), ignore_case=True)), key=str.lower)
13
14 def f2(data: list[str]) -> list[str]:
15     return list(filter(lambda f: str(f).lower().startswith('программист'), data))
16
17

```

Рис. 14 – реализация функций f1 и f2

```

19 def f3(data: list[str]) -> list[str]:
20     return list(map(lambda f: f + ' с опытом Python', data))
21
22
23 def f4(data: list[dict[Any]], sorted_data: list[str]) -> None:
24     number_prof = len(sorted_data)
25
26     salaries = gen_random(number_prof, 100000, 200000)
27
28     res = zip(sorted_data, salaries)
29
30     for item1, item2 in res:
31
32         print(f"{item1}, зарплата {item2} руб.")
33
34
35

```

Рис. 15 – реализация функций f3 и f4

```

(.venv) Vladimirs-Mac-mini:term3-PCPL-lab3-4 vladimir$ python process_data.py data_light.json
Программист с опытом Python, зарплата 104007 руб.
Программист / Senior Developer с опытом Python, зарплата 197658 руб.
Программист 1C с опытом Python, зарплата 126221 руб.
Программист C# с опытом Python, зарплата 119885 руб.
Программист C++ с опытом Python, зарплата 192587 руб.
Программист C++/C#/Java с опытом Python, зарплата 188290 руб.
Программист/ Junior Developer с опытом Python, зарплата 135244 руб.
Программист/ технический специалист с опытом Python, зарплата 176288 руб.
Программист-разработчик информационных систем с опытом Python, зарплата 175996 руб.
time: 0.0024487972259521484

```

Рис. 16 – демонстрация работы кода